1. **What are standalone components in Angular, and why were they introduced?**
    - **Standalone components in Angular are components that don't rely on Angular modules. They were introduced to simplify the structure and improve performance.**
2. **How does Angular handle component lifecycle hooks, and can you explain their order?**
    - **Angular handles component lifecycle hooks in this order: ngOnChanges, ngOnInit, ngDoCheck, ngAfterContentInit, ngAfter ContentChecked, ngAfterViewInit, ngAfterViewChecked, and ngOnDestroy.**
3. **What is the role of dependency injection in Angular, and how does it improve maintainability?**
    - **Dependency injection in Angular allows you to inject services into components, improving maintainability by promoting modular and reusable code.**
4. **Can you explain how Angular handles two-way data binding?**
    - **Angular handles two-way data binding using [(ngModel)], which binds data from the component to the view and vice versa.**
5. **How does Angular optimize rendering using change detection strategies?**
    - **Angular optimizes rendering using change detection strategies like Default and OnPush to minimize unnecessary updates.**
6. **How do you define routes in Angular, and what are some key configurations available?**
    - **You define routes in Angular using the RouterModule and configure paths, components, and guards.**
7. **Explain the difference between queryParams and route parameters. When would you use each?**
    - **queryParams are used for optional parameters in URLs, while route parameters are part of the URL path. Use queryParams for filters and route parameters for identifying resources.**

8. **How can lazy loading be implemented in an Angular application, and what benefits does it offer?**
    - **Lazy loading in Angular is implemented using the loadChildren property in route configuration, which loads modules only when needed, improving performance.**
9. **What are the differences between template-driven and reactive forms in Angular?**
    - **Template-driven forms are simpler and use directives, while reactive forms are more powerful and use FormControl and FormGroup classes.**
10. **How can you implement custom form validation in an Angular project?**
    - **Implement custom form validation by creating custom validators and using them in your form controls.**
11. **What is the purpose of form controls, and how do you manage form state effectively?**
    - **Form controls manage the value and validation state of form fields. Use FormGroup and FormControl to manage form state effectively.**
12. **How does TypeScript enhance Angular development? Can you provide examples?**
    - **TypeScript enhances Angular development with strong typing, classes, and interfaces. Example: defining a component with a class and using interfaces for type safety.**
13. **What are definite assignment assertions in TypeScript, and how can they help prevent initialization errors in Angular components?**
    - **Definite assignment assertions (!) in TypeScript ensure that a variable is initialized before use, preventing errors in Angular components.**
14. **What strategies would you use to handle errors in an Angular application?**
    - **Handle errors in Angular using try-catch blocks, global error handlers, and logging services.**
15. **How do Signals improve reactivity in Angular compared to traditional Observables?**
    - **Signals improve reactivity by providing a more efficient way to manage state changes compared to traditional Observables.**

16. **Explain how Angular supports progressive web apps (PWAs) and what features make an Angular app a PWA.**

    - **Angular supports progressive web apps (PWAs) with features like service workers, offline support, and push notifications.**

17. **What is the purpose of service workers, and how do they improve Angular application performance?**

    - **Service workers improve performance by caching assets and enabling offline functionality.**

18. **How does Angular's Ahead-of-Time (AOT) compilation work, and why is it useful?**

    - **Angular's Ahead-of-Time (AOT) compilation compiles code during the build process, reducing runtime errors and improving performance.**

19. **How would you debug an issue with improper imports in a standalone component?**

    - **Debug improper imports in a standalone component by checking import paths and ensuring dependencies are correctly referenced.**

20. **What tools and techniques would you use to profile and optimize Angular application performance?**

    - **Profile and optimize Angular application performance using tools like Angular DevTools, Chrome DevTools, and techniques like lazy loading and change detection strategies.**

    - **I have not used in my project but came to know about the tools**